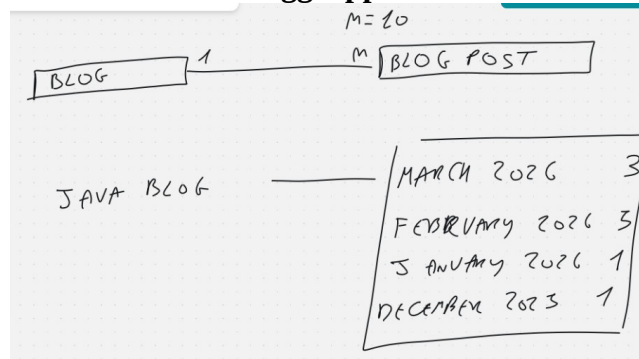


## Appunti del 12-13-26 Partizionamento e Raggruppamenti



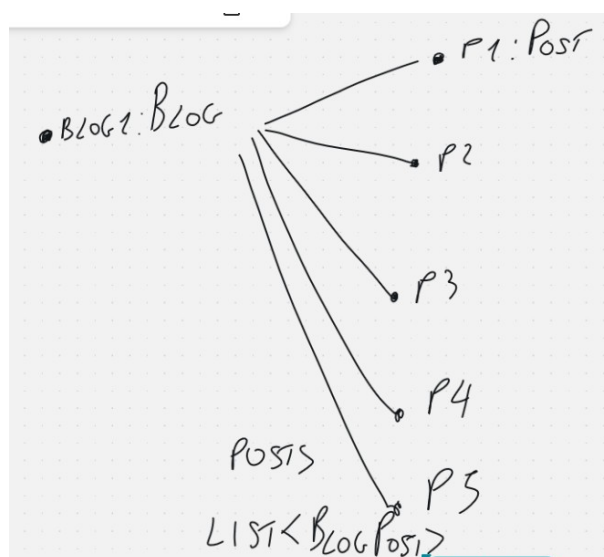
Il **partizionamento** (o raggruppamento) è una tecnica che si applica quando si ha una relazione 1 a molti tra entità, come nel caso di un Blog che contiene Blog Post.

Nell'esempio, un singolo Blog può avere fino a un massimo di 20 post ( $M = 20$ ). Invece di trattare tutti questi post come una lista piatta e indistinta, li si organizza in **gruppi logici** secondo un criterio — in questo caso il **mele di pubblicazione**.

Così, il Java Blog non espone semplicemente tutti i suoi post insieme, ma li presenta suddivisi: 3 post a marzo 2026, 3 a febbraio 2026, 1 a gennaio 2026, 1 a dicembre 2023. Ogni partizione rappresenta quindi un sottoinsieme coerente e delimitato dei dati.

Questo approccio porta diversi vantaggi pratici. Prima di tutto le **query diventano più efficienti**, perché invece di scorrere l'intera collezione di post si accede direttamente alla partizione di interesse. Inoltre i dati risultano più **organizzati e prevedibili** nella loro crescita, poiché ogni gruppo ha una dimensione naturalmente limitata. Infine, il partizionamento facilita operazioni come l'archiviazione o l'eliminazione di dati vecchi, che possono essere gestiti partizione per partizione anziché record per record.

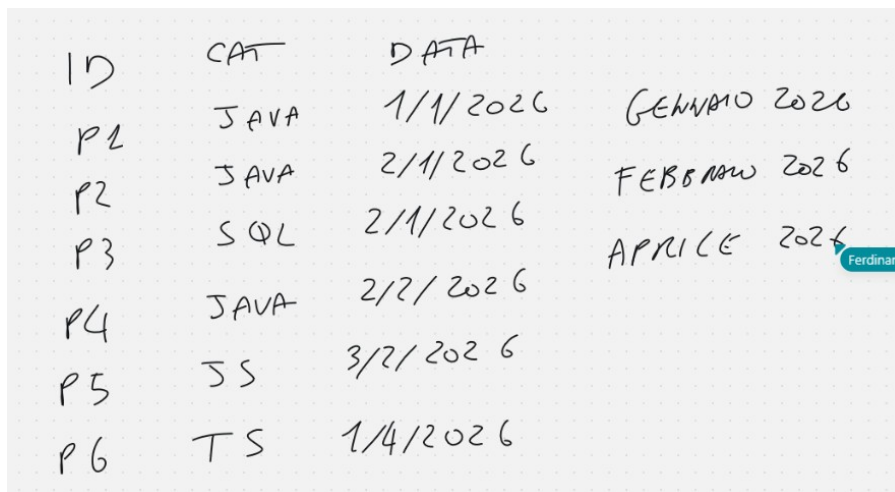
In sostanza, il raggruppamento è un modo per dare **struttura semantica** a relazioni che altrimenti sarebbero semplici liste, rendendo il modello dei dati più vicino al modo in cui gli utenti pensano e accedono alle informazioni.



**.Blog1**, che è un'istanza di tipo **Blog**, contiene un campo chiamato **posts** di tipo **List<BlogPost>**. Questo campo è una lista che punta direttamente a tutti i post collegati — **P1**, **P2**, **P3**, **P4**, **P5** — ciascuno dei quali è un'istanza di **BlogPost**.

In pratica si sta dicendo che all'interno dell'oggetto Blog esiste una collezione embedded o referenziata di post. Questo è l'approccio **senza partizionamento** tradotto in codice: il blog tiene traccia di tutti i suoi post tramite una singola lista piatta. Man mano che i post aumentano, questa lista cresce indefinitamente, il che può diventare problematico in termini di performance e gestione, specialmente nei database a documenti dove esistono limiti sulla dimensione dei documenti (come il limite di 16MB in MongoDB).

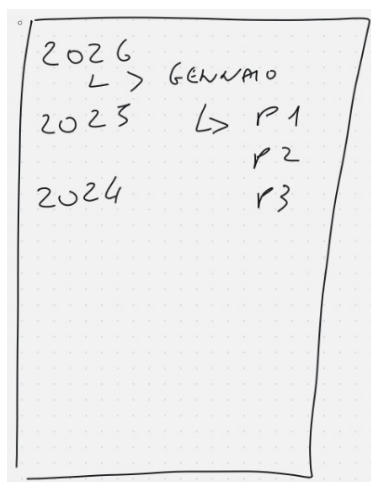
È esattamente per risolvere questo problema che si introduce il partizionamento: invece di una sola `List<BlogPost>` che cresce senza controllo, si suddivide in più gruppi gestibili, ciascuno con la propria lista limitata.



| ID | CAT  | DATA     |               |
|----|------|----------|---------------|
| P1 | JAVA | 1/1/2026 | GENNAIO 2026  |
| P2 | JAVA | 2/1/2026 | FEBBRAIO 2026 |
| P3 | SQL  | 2/1/2026 | APRILE 2026   |
| P4 | JAVA | 2/2/2026 |               |
| P5 | JS   | 3/2/2026 |               |
| P6 | TS   | 1/4/2026 |               |

Se si raggruppa **per mese** si ottengono 3 gruppi: Gennaio 2026 (P1), Febbraio 2026 (P2, P3, P4) e Marzo/Aprile 2026 (P5, P6). Questo raggruppamento riflette una dimensione **temporale** e tende a distribuire i post in modo naturale e prevedibile nel tempo.

Se invece si raggruppa **per categoria** si ottengono 4 gruppi: Java (P1, P2, P4), SQL (P3), JS (P5) e TS (P6). Questo raggruppamento riflette una dimensione **tematica** e ha senso quando l'utente naviga il blog per argomento.



Invece di raggruppare semplicemente per mese, si introduce prima un raggruppamento per **anno** (2026, 2023, 2024) e poi, all'interno di ogni anno, un ulteriore raggruppamento per **mese** (es. Gennaio). Solo a quel punto si trovano i singoli post (P1, P2, P3).

Si parte da una lista piatta di post, dove tutti gli elementi sono allo stesso livello e vengono attraversati in sequenza, e la si trasforma in una lista di liste, dove ogni elemento della lista esterna non è più un singolo post ma un gruppo che contiene al suo interno una lista di post appartenenti allo stesso periodo.

Nel caso del partizionamento gerarchico per anno e mese, questa idea si estende ulteriormente: la lista esterna contiene gli anni, ciascun anno contiene una lista di mesi, e ciascun mese contiene la lista effettiva dei post. Si passa quindi da una struttura piatta a una struttura annidata a più livelli di profondità.

Il vantaggio pratico è che invece di dover scorrere tutti i post uno per uno per trovare quelli di interesse, si naviga la gerarchia scendendo di livello in livello — prima si sceglie l'anno, poi il mese, poi si accede ai post — lavorando ogni volta su un insieme molto più piccolo e delimitato di dati.

## Piccolo Ripasso

MySQL è un **sistema di gestione di database relazionale** (RDBMS), uno dei più diffusi al mondo, basato sul linguaggio SQL (Structured Query Language).

### I fondamentali

I dati in MySQL sono organizzati in **tabelle**, ciascuna composta da righe e colonne. Ogni colonna ha un tipo di dato definito (intero, stringa, data, ecc.) e ogni riga rappresenta un record. Le tabelle sono contenute all'interno di un **database**, che funge da contenitore logico.

### Operazioni base

Le operazioni fondamentali si esprimono tramite quattro istruzioni principali, spesso chiamate CRUD: **INSERT** per inserire dati, **SELECT** per leggerli, **UPDATE** per modificarli e **DELETE** per eliminarli.

### Chiavi e relazioni

Un concetto centrale è la **chiave primaria** (PRIMARY KEY), un campo che identifica univocamente ogni riga di una tabella. Per collegare due tabelle si usa invece la **chiave esterna** (FOREIGN KEY), che fa riferimento alla chiave primaria di un'altra tabella, realizzando così le relazioni tra entità.

### Vincoli e integrità

MySQL permette di definire vincoli sui dati, come NOT NULL per impedire valori vuoti, UNIQUE per garantire che un valore non si ripeta, e CHECK per validare condizioni specifiche. Questi vincoli garantiscono la **integrità referenziale** del database.

### Indici

Gli **indici** sono strutture ausiliarie che velocizzano le query di ricerca. Funzionano in modo simile all'indice di un libro: invece di scorrere tutte le righe, il database salta direttamente alle righe di interesse. Il rovescio della medaglia è che gli indici occupano spazio e rallentano leggermente le operazioni di scrittura.

### Transazioni

MySQL supporta le **transazioni**, ovvero sequenze di operazioni che vengono eseguite come un blocco atomico: o vanno tutte a buon fine, o nessuna viene applicata. Questo garantisce la consistenza dei dati anche in caso di errori o interruzioni, seguendo il principio ACID (Atomicità, Consistenza, Isolamento, Durabilità).

```

2
3 • create table blog_post
4 (
5     id int primary key auto_increment,
6     title varchar(100),
7     content varchar(100),
8     publishedOn datetime,
9     blog_id int,
10    foreign key(blog_id) references blog(id)
11    on delete restrict
12    on update cascade

```

**ON DELETE RESTRICT** significa che se si tenta di eliminare un record nella tabella padre, MySQL **blocca l'operazione** nel caso in cui esistano record figli che lo referenziano. In pratica il database si rifiuta di cancellare il padre finché esistono figli che dipendono da lui, proteggendo così l'integrità referenziale. Ad esempio, se si prova a eliminare un blog che ha ancora dei post collegati, MySQL restituisce un errore e non esegue la cancellazione.

**ON UPDATE CASCADE** significa invece che se si modifica la chiave primaria di un record padre, MySQL **propaga automaticamente la modifica** a tutti i record figli che la referenziano. Quindi se l'ID di un blog cambia da 1 a 99, tutti i post che avevano `blog_id = 1` vengono aggiornati automaticamente a `blog_id = 99`, senza bisogno di farlo manualmente.

```

• SELECT
    year(published_on) as year ,
    month(published_on) as month,
    count(*)
FROM
    voices.blog_post
where
    blog_id = 1
group by
    year(published_on),
    month(published_on)

```

Il query dice: seleziona l'anno di pubblicazione, il mese di pubblicazione e il numero di post, prendendoli dalla tabella `blog_post` del database `voices`, considerando solo i post che appartengono al blog con ID uguale a 1, e raggruppalì per anno e per mese. `count (*)` conta **quante righe ci sono in ciascun gruppo**. Quindi, una volta che i post sono stati raggruppati per anno e mese, conta quanti post ricadono in quel determinato mese di quell'anno. Il simbolo `*` significa "conta tutte le righe, senza escludere nulla".

```

1 package com.generation.voices.model;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class PostsByYear {
7
8     // i mesi sono un vettore di liste di post
9     int year;
10    PostsByMonth[] months = new PostsByMonth[12];
11
12    public PostsByYear(int year)
13    {
14        this.year = year;
15        for(int i=0; i<12; i++)
16            months[i] = new PostsByMonth(i+1, year);
17    }
18
19 }
20
21

```

Il costruttore di `PostsByYear` riceve l'anno, lo salva, e poi inizializza tutti e 12 i mesi con un ciclo for che crea un oggetto `PostsByMonth` per ciascuno, passando il numero del mese ( $i+1$ , quindi da 1 a 12) e l'anno. Al termine del costruttore si ha un array completamente popolato con le 12 partizioni mensili, pronte per ricevere i post.

```

7
8 public class Blog {
9
10    int id;
11    String title;
12    List<BlogPost> posts = new ArrayList<BlogPost>();
13
14
15    public Map<Integer, PostsByYear> getPostsByYearAndMonth()
16    {
17        int minYear = 4500;
18        int maxYear = 0;
19
20        for(BlogPost p:posts)
21        {
22            if(p.publishedOn.getYear()<minYear)
23                minYear = p.publishedOn.getYear();
24            if(p.publishedOn.getYear()>maxYear)
25                maxYear = p.publishedOn.getYear();
26        }
27
28        Map<Integer, PostsByYear> res = new LinkedHashMap<Integer, PostsByYear>();
29        for(int year=minYear; year<=maxYear; year++)
30            res.put(year, new PostsByYear(year));
31
32        for(BlogPost p:posts)
33        {
34            // devo capire dove vanno
35            int year = p.publishedOn.getYear();
36            int month = p.publishedOn.getMonthValue()-1;
37            res.get(year).months[month].posts.add(p);
38        }
39
40        return res;
41    }
42

```

Il metodo fa tre cose in sequenza.

**Prima** scorre tutti i post per trovare l'anno più vecchio e l'anno più recente tra le date di pubblicazione, in modo da sapere quale intervallo di anni coprire.

**Poi** crea una mappa e la popola con un `PostsByYear` vuoto per ogni anno nell'intervallo trovato. Se i post vanno dal 2020 al 2026, la mappa avrà 7 entry, una per ogni anno, ciascuna già inizializzata con i suoi 12 mesi vuoti.

**Infine** scorre di nuovo tutti i post e per ciascuno capisce dove deve andare: prende l'anno e il mese dalla data di pubblicazione, naviga nella mappa fino alla posizione giusta e inserisce il post lì. La riga `res.get(year).months[month].posts.add(p)` riassume tutta questa navigazione in una sola istruzione: vai all'anno, vai al mese, aggiungi il post.

Il risultato finale è la lista piatta di post trasformata nella struttura gerarchica partizionata per anno e per mese, che era esattamente l'obiettivo.

Al frontend deve arrivare la mappa che esce da `getPostsByYearAndMont`